159.102 Notes

Section 4 : Random Numbers + CPU and Machine Instructions

Random Numbers

Random numbers are used in many simulations. It is important to know how to use random numbers (particularly in games and simulations) and it is also important to know that random numbers generated by a computer are not actually random. Unless you use a different random number seed, you will get exactly the same sequence of random numbers again – which means that the numbers are not truly random. With truly random numbers it would be impossible to predict the next number in a sequence or to repeat a sequence.

True random numbers

Measure some physical phenomenon that is expected to be random. For example, atmospheric noise, cosmic background radiation, some types of radioactive decay, etc. While this method can produce truly random numbers (if set up properly) it is usually rather slow as there are pauses while the system waits for the next random signal to be generated. This method is outside the realm of Computer Science.

Pseudo-random numbers

A sequence of apparently random numbers that are determined by an initial value (or seed). If the same seed is used again, the identical sequence of pseudo-random numbers is produced again. They are much faster to produce than true random numbers and there are times when it is useful to exactly reproduce a previous sequence of random numbers.

Note: the discussion below is entirely about pseudo-random number generators

An example random number generator

Here is the code needed to make your own random number generator:

```
#include <iostream>
#include <iomanip>
using namespace std;

unsigned int GetUint();
double GetUniform();

unsigned int mw, mz;  // must be global

int main() {
  float f;
  int count;
  char choice;
  // load mw and mz - these two numbers make up the seed
  mw = 35;
  mz = 478;
```

```
  count = 0;
  while (count < 50) {
    f = (float) GetUniform();
    f = f * 100;
    cout << setprecision(2) << fixed << f << ", ";
    count++;
  }
  cout << endl;

  cout << "\nDo you want a new seed? (Y/N): ";
  cin >> choice;
  if ((choice == 'Y') || (choice == 'y')) {
    mw = 9367;   // change the seed
    mz = 872;
  } else {
    mw = 35;    // start with the same seed again
    mz = 478;
  }
  count = 0;
  while (count < 50) {
    f = (float) GetUniform();
    f = f * 100;
    cout << setprecision(2) << f << ", ";
    count++;
  }
  cout << endl << endl;
}

unsigned int GetUint() {
  mz = 36969 * (mz & 65535) + (mz >> 16);
  mw = 18000 * (mw & 65535) + (mw >> 16);
  return (mz << 16) + mw;
}

double GetUniform() {
// returns a double in the open interval (0, 1)
  unsigned int u;
  u = GetUint();
  return (u + 1.0) * 2.328306435454494e-10;
}
```

Note 1: the GetUniform function produces uniform random numbers in the interval (0, 1)
Note 2: this is risky as implementations may change – rather use a provided random function

Using bitwise operations

Bitwise operations and shifting are the core of a random number generator.
Note – you are NOT expected to understand the code in the random number generator.
It is, however, interesting to note that bitwise operations are required to make it work.

Uniform Random Number Generators

Most random number generators are uniform. This means that the random numbers are generated uniformly across the range from 0 to 1. In other words, every number in the range is equally likely.

However, there may be times when you need random numbers that have a normal distribution or an exponential distribution or any of a long list of possible distributions.

Random number generators that are not uniform are usually based on uniform random number generators. Thus the uniform random number generator is the most important. In our discussions here, we will use only uniform random number generators.

Testing Random Number Generators

Many different tests (usually statistical tests) exist to check if a random number generator is a good one, in other words, if it produces a long sequence of pseudo-random numbers that are actually close to being random. Always use a random number generator that has been fully tested. Otherwise, you may find that your sequence of random numbers is not actually random – and this can cause problems. **Hint:** do *not* use the code listed above as it has not been fully tested!

Boundary Values

It is important to know the exact range of the random numbers that are produced. Usually random number generators will produce `double` values from 0 to 1. But does this mean that 0 and 1 are included or not included in the range? Three possibilities are:
The **open interval** (0, 1) which means that 0 is not part of the range and 1 is not part of the range
The **closed interval** [0, 1] which means that 0 is included in the range and 1 is included in the range
The **half-open interval** [0, 1) which means that 0 is included but 1 is not part of the range

Random Number Seed

A seed is a number that is used to start the sequence. It usually consists of one number but sometimes there can be two numbers, for example, see the code above. If you run the program again with the same seed, it will produce exactly the same random sequence. Sometimes this is useful, for example, if I am testing a new game (or climate simulation), I can generate the same sequence of random numbers to see if the game (or simulation) now reacts in a different way.

In other programs, it is better to have a different random sequence generated each time, for example, in computer games and especially in cryptography. One solution for this problem is to base the seed on a value that keeps changing, for example, the current time is often used as a seed.

Random Numbers are used in Cryptography

Data is usually encrypted and decrypted using known algorithms. The encrypted data sent between sender and receiver is also known (or could be known). Therefore the security of the whole system depends on the key. Knowledge of the key enables the decryption of the encrypted data. Encryption keys must be unpredictable and difficult (hopefully impossible) for an attacker to calculate. Thus the keys (on which everything else depends) must be random.

Algorithms to produce encryption keys make use of both “true” random numbers and pseudo-random numbers. The “true” random number is used as the seed for a pseudo-random number sequence. To avoid possible cyber-attacks, it is important to include “true” random numbers at some point (even if only as a seed), otherwise a potential hacker may be able to work out the pseudo-random sequence used by the algorithm.

Random Numbers are used to simulate probabilistic events

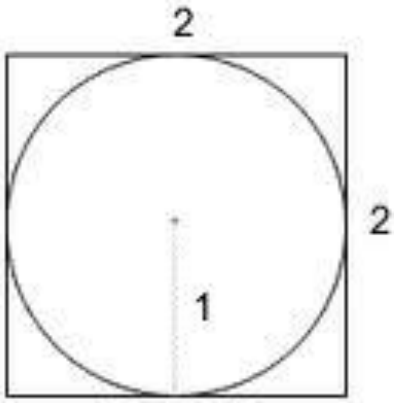
For example, flipping a coin, throwing dice, guessing the top card in a deck of cards, etc…

Set up a simulation (e.g. flipping a coin).
Run the simulation many thousands of times.
The results will be a good approximation of the probability of getting heads or tails, etc.

Random Numbers are used in Monte Carlo methods (in statistics)

Monte Carlo methods are a class of computational algorithms that rely on repeated random sampling to obtain results. It is difficult to give a general and precise definition that suits all Monte Carlo methods, but in general they tend to follow this pattern:

1. Define a domain of possible inputs
2. Generate inputs randomly over the domain
3. Perform a deterministic computation on the inputs
4. Aggregate (combine/average/total) the results



For example, find an approximate value for pi.

1. Draw a square and then draw a circle inside the square and touching it at four points. Note that the radius of the circle is r.
Area of circle = pi * r^2
Area of square = 2r * 2r = $4r^2$
So (Area of circle) / (Area of square) = (pi * r^2) / (4 * r^2) = pi / 4

2. Generate points randomly inside the square. N = total number of points.
M = number of those points that are also in the circle. If you generate many points, then N is an approximation to the area of the square and M is an approximation to the area of the circle.

3. M / N = pi / 4 means that pi = (4 * M) / N

4. How good is this value for pi? Can we improve it?

Monte Carlo methods are often useful but they are slow because a large number of random numbers has to be generated.

Random Numbers are used in calculating chance events

Random numbers are very useful in simulations such as predator-prey models.
For example, what is the chance of a predator (fox) catching prey (a rabbit)?

First approach – lots of detail and no random numbers:
Take all factors into account, e.g. speed of animals, fitness of animals, weather, terrain, etc.
Set up a complicated formula. Calculate if the fox catches the rabbit.

Second approach – chance:
The fox has a 56% chance of catching a rabbit.
Generate a random number from 1 to 100. If the number is 56 or less, then the fox catches the rabbit.
Advantages of this second approach:
1) very simple – does not require detailed data (often unknown) and a complicated formula
2) modify the chance until it feels right – maybe 48% chance or maybe 60% chance, etc
3) produces surprisingly realistic results

Random Numbers in Games

Random numbers (generated by throwing dice) have been used by people since at least 3000BC. Random numbers are heavily used in computer games, very often to calculate chance events (as described above). For example, two characters in D&D game may attempt to pick a lock. Character A has lock pick skill of 85% (a good chance of success). Character B has lock pick skill of 17% (a very low chance of success). But character A may still fail and character B may succeed – it's all up to chance (and random numbers). There are also some games that do not use random numbers, for example there are no random events in a game of chess. There may be luck (you might make a really good move without realising it) but there are no random events.

Writing programs that use random numbers

Do not use the random generator shown above. This was shown just to illustrate the use of bitwise operations to make the random number generator work. However, this random number generator has not been fully tested so best not to use it. Use the provided C++ functions (see below).

A program that uses random numbers

Here is a program that checks to see if a certain event occurs. Always check your program to see if it is working reasonably well – that is what `count` is doing in the program.

```
#include <iostream>
#include <cstdlib>    // need this for rand and srand
using namespace std;

int RandomInt(int first, int last);

//-----------------------------------------------------------------
int main() {
  int chance, count, i;

  srand(683);  // the seed - what happens if you change it?

  cout << "This event has a 56% chance of occurring.\n\n";
  count = 0;
  for (i = 0; i < 1000; i++) {
    chance = RandomInt(1, 100);
    if (chance <= 56) {  // chance of event occuring
         count++;
      }
  }
  cout << "The event occurred " << count << " times.\n";
}

//-----------------------------------------------------------------
int RandomInt(int first, int last) {
// returns a random number in the range [first, last]
  int diff;
  diff = (last - first) + 1;
  return rand() % diff + first;
}
```

The CPU and Machine Instructions

The Central Processing Unit (CPU)

The computer has two main sections – the **memory** and the **Central Processing Unit** (CPU). Both of these contain circuits such as flip-flops, adders, etc.

The CPU is divided into three sections as follows:

- Control Unit – circuits to coordinate the activities of the computer
- Arithmetic Logic Unit (ALU) – circuits to manipulate data (e.g. adder, complement, shift)
- Registers – memory used as temporary holding areas for data being manipulated by the ALU. Registers feed directly into the ALU circuits whereas general memory does not.

Bus

The CPU is connected to the memory by a collection of wires called the **bus**. The CPU uses the bus to transfer data between the memory and the registers.

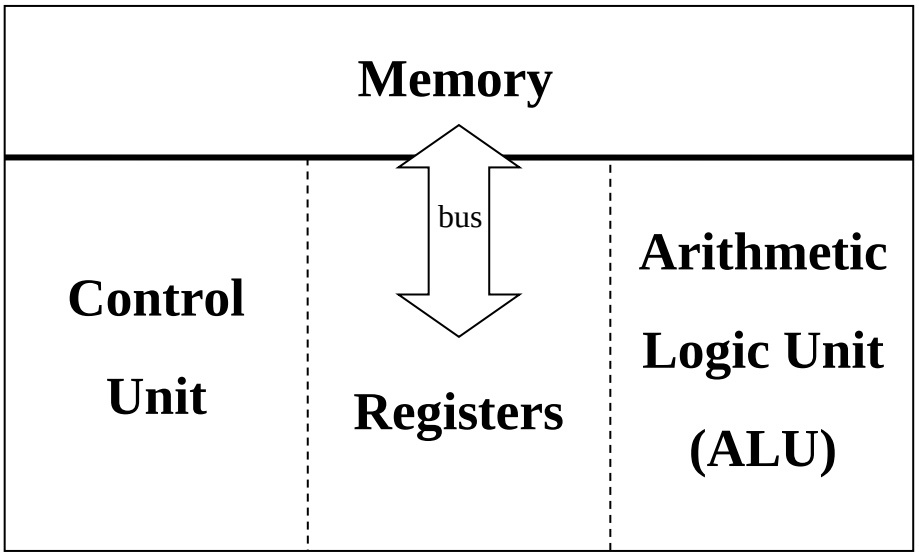
For example, the following steps are required to add two numbers together:

Step 1: get the first value from memory and load it into register 1.

Step 2: get the second value from memory and load it into register 2.

Step 3: activate the adder with these two registers as input and load the result into another register.

Step 4: store the contents of the other register (the result) into a memory location.



The **CPU** consists of the Control Unit + Registers + ALU

Memory

Memory consists of many memory locations where each location is a group of bits (circuits) that can represent a binary number. Each memory location has an **address** (similar to an index in an array). Memory only stores numbers. However, we can look at the numbers in two different ways as follows:
a) we can treat the numbers as numbers (i.e. as data)

b) we can treat the numbers as machine instructions
In fact, sometimes we treat numbers in memory as both data and instructions. When a program starts running, it looks at the number sitting in a specified location in memory (we use location 0). The CPU treats this number as the first machine instruction of the program.

Machine Instructions

The machine contains circuits that allow it to perform certain instructions. These are known as the machine instructions. They have nothing to do with instructions in a high-level language such as C++. Such languages require a **compiler** to convert the high-level program into a **machine code program**. The machine code program is usually stored in a file with the extension **.exe**. Machine instructions can be divided into three groups:

- Data Transfer Instructions
- Arithmetic/Logic Instructions
- Control Instructions

Data Transfer Instructions

Instructions that request the movement of data from one location to another. Generally this involves making a copy of the data in the new location – it is not erased from the initial location. We could also include input and output (I/O) instructions in this group.

Examples:

LOAD = transfer data from a memory location to a register

STORE = transfer data from a register to a memory location

COPY = transfer data from a register to another register (also called MOVE)

Arithmetic/Logic Instructions

Instructions that activate circuits in the ALU.

Examples:

ADD, AND, OR, XOR, SHIFT, NEGATE (complement and add 1)

Control Instructions

Instructions that direct the execution of the program. Without control instructions, the computer would always execute instructions ***sequentially***. Control instructions alter the **flow of control**.

Examples:

JUMP to line 25 *an **unconditional** jump*

IF x < 3 JUMP to line 9 *a **conditional** jump*

RETURN

HALT

Stored Programs

Early computers had **fixed programs**. Some simple computers still use this design, for example a calculator can do arithmetic but it cannot do word processing. To change the program of such a

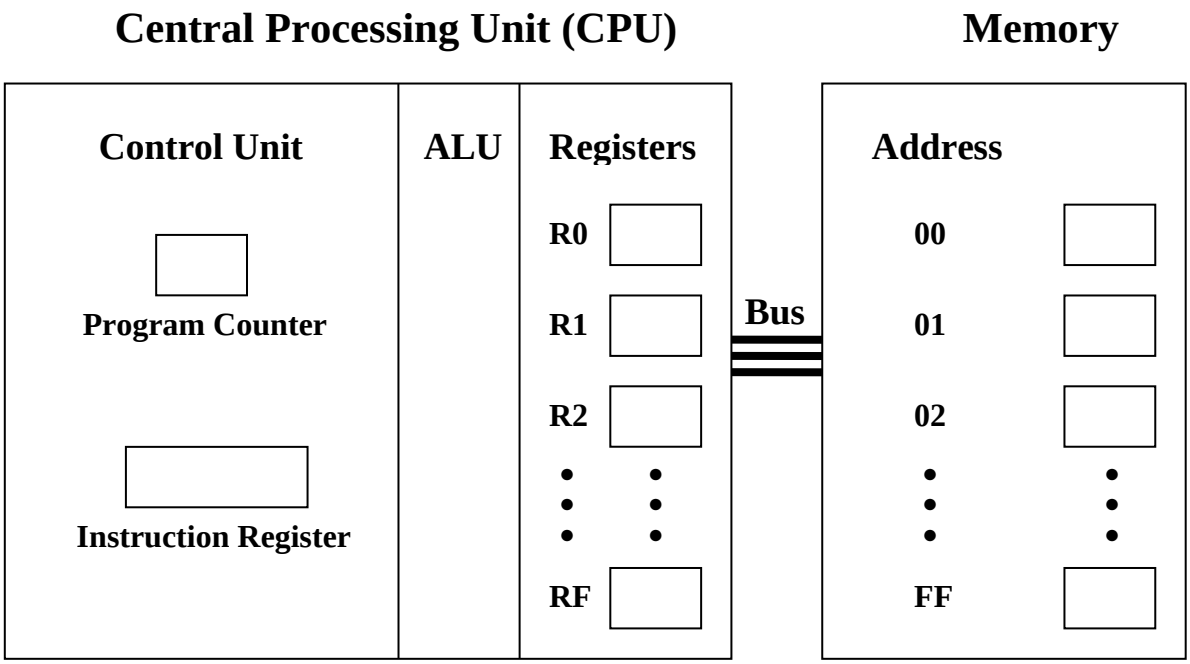
machine you have to rewire it or even redesign it. The first computers were designed for very specific tasks, such as calculating the trajectories of artillery shells in World War 2.

The **stored program** changed all that by making computers much more flexible. Each machine instruction is represented as a binary number and the numbers are stored in memory. Then the Control Unit can be set up to progress through the instructions and execute each one in turn. These numbers representing instructions are called **machine code**. Each instruction is a binary number consisting of two sections: the **opcode** and the **operands**.

The **opcode** identifies the instruction.
The **operands** are further information, such as memory locations, register numbers, etc.

The stored program concept also allows programs to **modify themselves while they are running**. Instructions are binary numbers stored in memory. Data are also binary numbers stored in memory. Thus it is possible to read in data and then execute it as an instruction. Most high-level languages, such as C++, do not allow us to modify the program while running. However, there are a few languages, such as Lisp, that do allow this. Unfortunately, the stored program concept means that programs can be altered by hackers if they gain access to the computer memory.

The MASSY Machine – a simple computer



This diagram is a good overview of the basics of any computer. Some computers have multiple processing units (and more memory) but our main aim is to gain an understanding of how a computer processes instructions. Practicing machine code programming on a real computer is a problem because (a) every computer has a different set of machine instructions, and (b) most modern computer systems are very complicated. We will practice programming on the MASSEY machine (as shown in the diagram above). This is not a real computer but gives a good feel of what it is like to work with machine code.

Memory in the MASSEY machine consists of 256 locations. Each location has an address numbered from 0 to 255 (in binary). We use hexadecimal as a shorthand for binary so we number the memory locations from 00 to FF. Each memory location consists of 16 bits (2 bytes). Total memory is thus 512 bytes (= 0.5 KB).

In the CPU we have 16 registers numbered from 0 to 15. Again we number them in hexadecimal from 0 to F. We usually call them R0 to RF (or R15) just to distinguish them from memory locations. Each register is the same size as a memory location and consists of 16 bits.

Machine instructions consist of 16 bits and thus each instruction occupies one memory location. The first instruction always occupies memory location 0, the next instruction occupies memory location 1 and so on. Data values (if needed) are stored any where in memory after the instructions. Warning! Sometimes data and instructions can become mixed up – they are all binary numbers after all.

In the Control Unit we have the **program counter (PC)** which is memory consisting of 16 bits. It always contains the ***address of the next instruction***. It is initialised to zero, thus the instruction sitting in memory location zero is always the first instruction to be executed.

The Control Unit also includes the **instruction register** which is memory consisting of 16 bits and it always contains the current instruction.

Like any computer the MASSEY machine can work with integers or with floating point values. However, in order to keep our lives simple, we will only work with integers.

The Machine Cycle

1. Fetch

– look in the program counter (PC) and get the address of the next instruction.

– get the instruction (number) from that memory location and load it into the instruction register.

– add 1 to the program counter

2. Decode

– analyse the instruction in the instruction register and "decide what needs to be done".

3. Execute

– activate the correct circuits to perform the instruction. Note that during execution, numbers may be transferred to/from memory and the PC may be modified (by jump instructions).

Repeat this cycle until a HALT instruction is executed.

Extra note for interest only – this is NOT part of our course

Look at the **bus** in the diagram above. This separation between CPU and memory leads to the ***von Neumann bottleneck*** (named after John von Neumann, a scientist who worked on many projects including the atom bomb). In modern machines the ***data transfer rate*** (along the bus between memory and registers) is much less than the rate at which the CPU can work. This seriously limits the effective processing speed when the CPU is working on large amounts of data as the CPU is continuously forced to wait for data to be transferred. More information here (for interest only):
https://en.wikipedia.org/wiki/Von_Neumann_architecture

The Instruction Set for the MASSEY Machine

Each instruction consists of 16 bits written as 4 hexadecimal digits. The first digit is the **opcode**. The other 3 digits are the **operands**. The list of machine instructions follows.

1RXY **Load** register R with the value XY. (Note 2-digit hex limit on number size)
e.g. 1213 means load register R2 with the hexadecimal number 13 (19 in decimal).

20RS **Load** register R with the number in register S.
e.g. 20A7 means load register R10 with the number in register R7.

3RXY **Load** register R with the number in the memory location at address XY.
e.g. 34AB means load register R4 with the contents from memory location AB

4RXY **Store** the number in register R in the memory location at address XY.
e.g. 45B1 means store the contents of register R5 in the memory location B1

5RST **Add** the numbers in registers S and T and load the result into register R.
Note: this is **floating point addition**. (we do not use this instruction)
e.g. 534E means add the numbers in R4 and R14 and load the result into R3.

6RST **Add** the numbers in registers S and T and load the result into register R.
Note: this is **integer addition** using 2's complement arithmetic.
e.g. 6823 means add the contents of R2 and R3 and load the result into R8.

70R0 **Negate** register R. (Carry out complement and add 1 – in effect, multiply R with -1)
e.g. 70A0 means multiply the number in R10 by -1.

8RXY **Shift** the number in register R to the **right** for XY bits
e.g. 8403 means that the number in R4 must be shifted 3 bits to the right.

9RXY **Shift** the number in register R to the **left** for XY bits
e.g. 9602 means that the number in R6 must be shifted 2 bits to the left.

ARST **AND** the numbers in registers S and T and load the result into register R.
e.g. A045 means AND the numbers in R4 and R5 and load the result into R0.

BRST **OR** the numbers in registers S and T and load the result into register R.
e.g. BC74 means OR the numbers in R7 and R4 and load the result into R12.

CRST **XOR** the numbers in registers S and T and load the result into register R.
e.g. C5F3 means XOR the numbers in R15 and R3 and load the result into R5.

DRXY **Jump** to the instruction at address XY **`if`** the value in register R is equal to the value in register R0.
e.g. D43C means the following: (a) compare the contents of R4 with R0.
(b) if they are equal load 3C into the **program counter**.

E000 **Halt**

Example program 1

1102	Load register R1 with 2
1203	Load register R2 with 3
6012	R0 = R1 + R2
40FF	Store register R0 in memory location FF
E000	Halt

Walk through this program showing the contents of the registers at each step.

Note that this list of machine instructions is equivalent to the C++ code: `x = 2 + 3;`

Example program 2

What is the following program doing?:

Address	Contents
00	1101
01	300B
02	D104
03	E000
04	1205
05	7020
06	310C
07	6312
08	430C
09	E000
0A	0000
0B	0001
0C	000C

First of all, write down the instructions with an explanation of each:

1101	Load register R1 with 1
300B	Load register R0 with the value in memory[0B]
D104	Jump to the instruction stored at memory[04] if R0 == R1 (i.e. if R0 contains 1)
E000	Halt (this only happens if no jump occurred)
1205	Load register R2 with 5
7020	Negate value in register R2
310C	Load register R1 with the value in memory[0C]
6312	R3 = R1 + R2
430C	Store register R3 in memory[0C]
E000	Halt

Then work through the program. At each step write down the contents of the registers, the program counter, the instruction register and memory[0C].

Note that this machine code program is equivalent to the C++ code:

```
if (result == 1) { x = x - 5; }
```

where result is the variable at memory[0B] and x is the variable at memory[0C].